Somaiya Vidyavihar University

K. J. Somaiya College of Engineering, Mumbai

Operating System

Module 1. Introduction to System software

Dr. Prasanna Shete
Dept. of Computer Engineering
prasannashete@somaiya.edu
Mobile/WhatsApp 9960452937

SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

Module 1: Introduction to System software

- Concept, introduction to various system programs such as assemblers, loaders, linkers, macro processors, compilers, interpreters, operating systems, device drivers
- Operating System Objectives and Functions,
- The Evolution of Operating Systems
- Operating system structures
- OS Design Considerations for Multiprocessor and Multicore architectures
- System Calls
- Linux Kernel and Shell
- System boot

Somaiya TRUST

Operating System

- Entity that makes a computer system operational
- Interface between user and computer applications and hardware
- *OS is a system software that controls execution of programs and acts as an interface between applications and computer hardware*

Operating System Objectives

- Convenience
  - Convenient to use machine
- Efficiency
  - Efficient use of system resources
- Ability to evolve
  - Permits effective development, testing & introduction of new system functionalities without interfering with service

Services provided by OS

- Program Development
  - Utility programs; Editors and debuggers
- Program Execution
  - Loading of instructions & data into main memory; initialization of I/O and files etc.
- Access to I/O devices
  - OS hides the details of instructions/control signals of I/O
- Controlled Access to files
  - Size of file, free space; protection in multiple user scenario
- System Access
  - Authorized use in public domain
- Error detection and response
  - Memory error; device failure/malfunction
- Accounting
  - Usage statistics of resources; useful in future enhancements; billing (multi-user scenario)

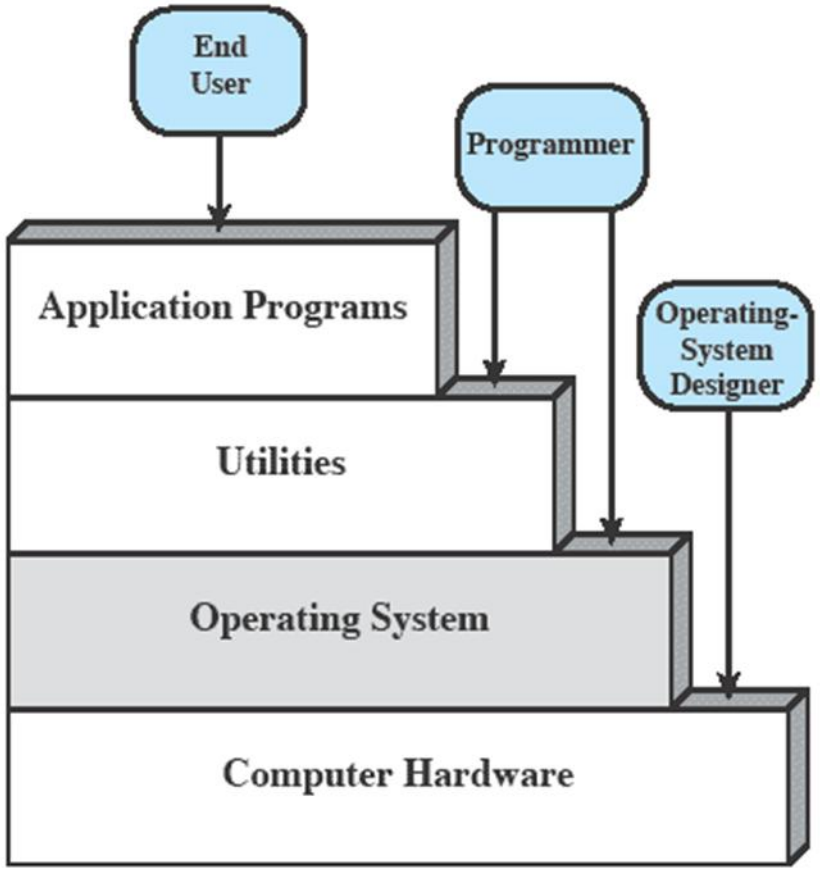
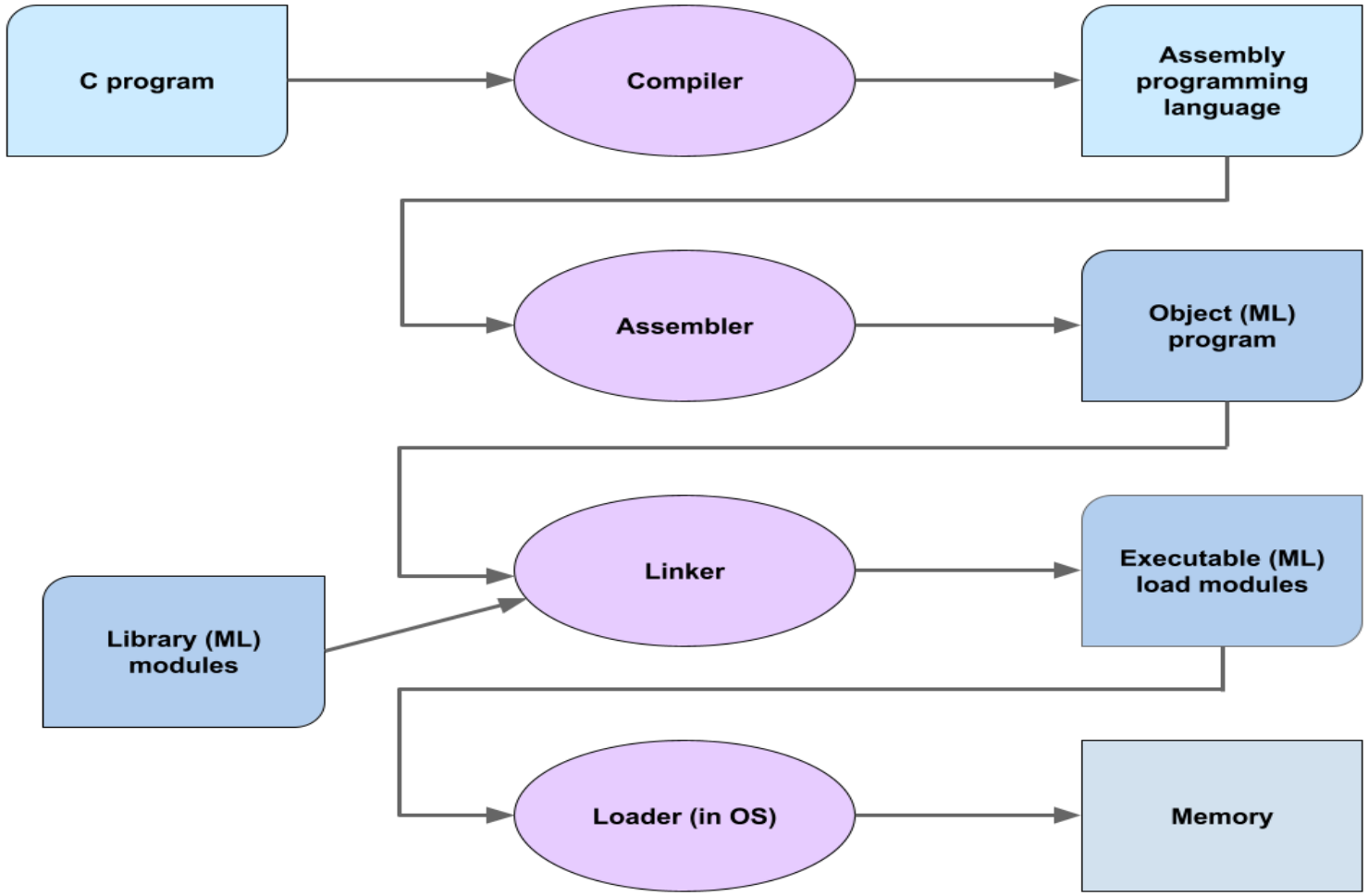


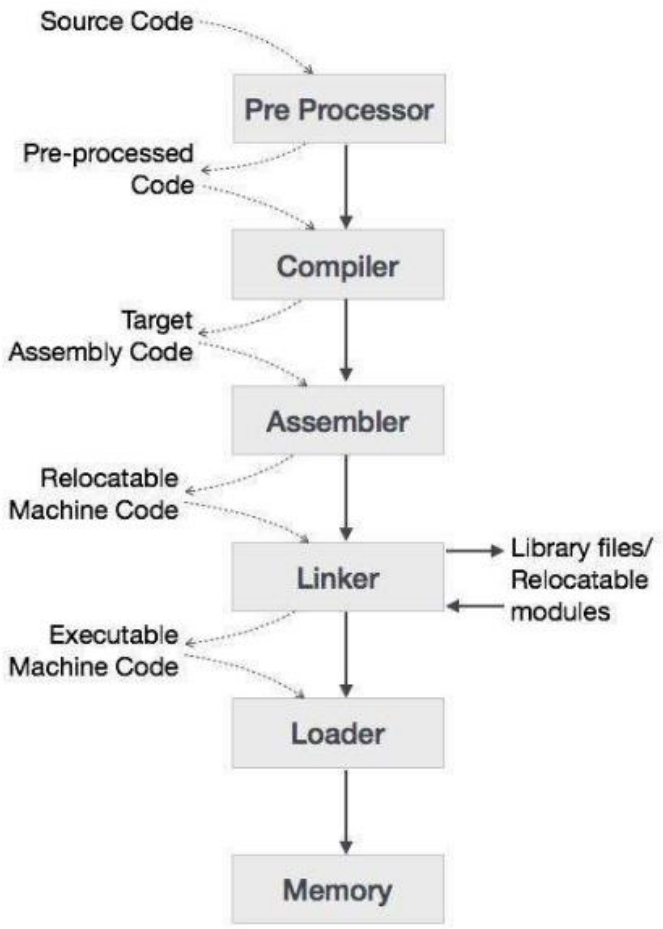
Figure 2.1 Layers and Views of a Computer System

How computers translate HLL (C) to ML format



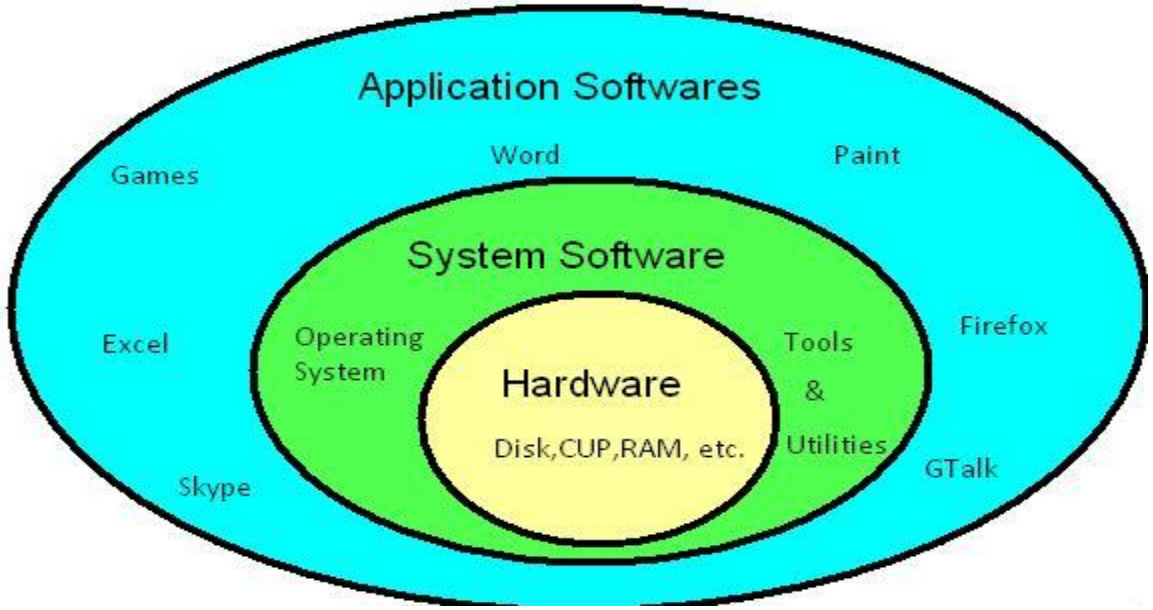
How a program, using C compiler, is executed on a host machine ?

- User writes a program in C language (high-level language)
- The C compiler, compiles the program and translates it to assembly program
- An assembler then translates the assembly program into machine code (object file)
- A linker is used to link all the parts of the program together for execution (executable machine code)
- A loader loads all of them into memory and then the program is executed



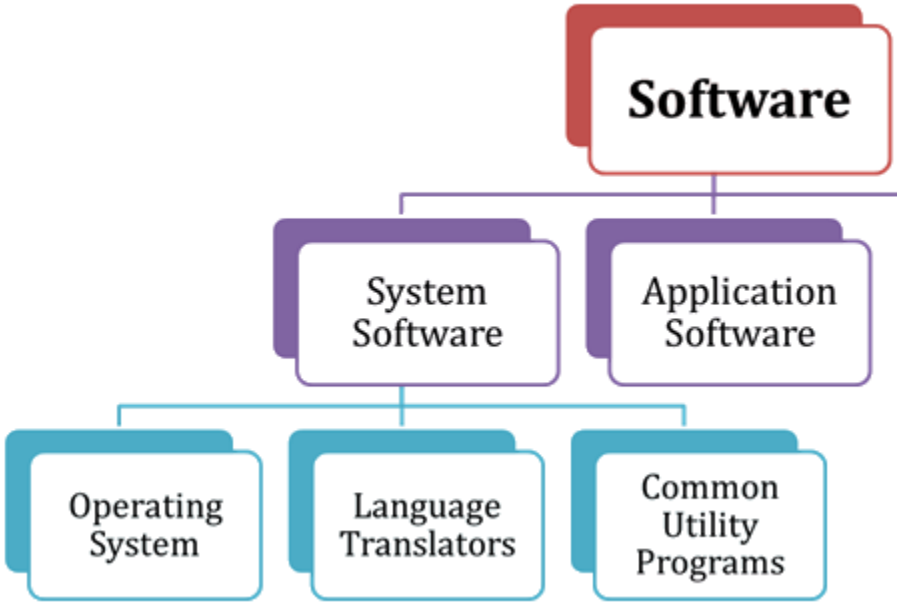
System Programming

- **System Programming** can be defined as act of building Systems Software using System Programming Languages
- Hierarchy- last is Hardware, then System Programs, and finally Application Programs
- System software serves as the interface between the hardware and the end users



System Programming

- Two main types of software: system software and application software
  - System software includes the programs that are dedicated to managing the computer itself
  - System Software is a set of programs that controls and manages the operations of computer hardware; It also helps application programs to execute correctly



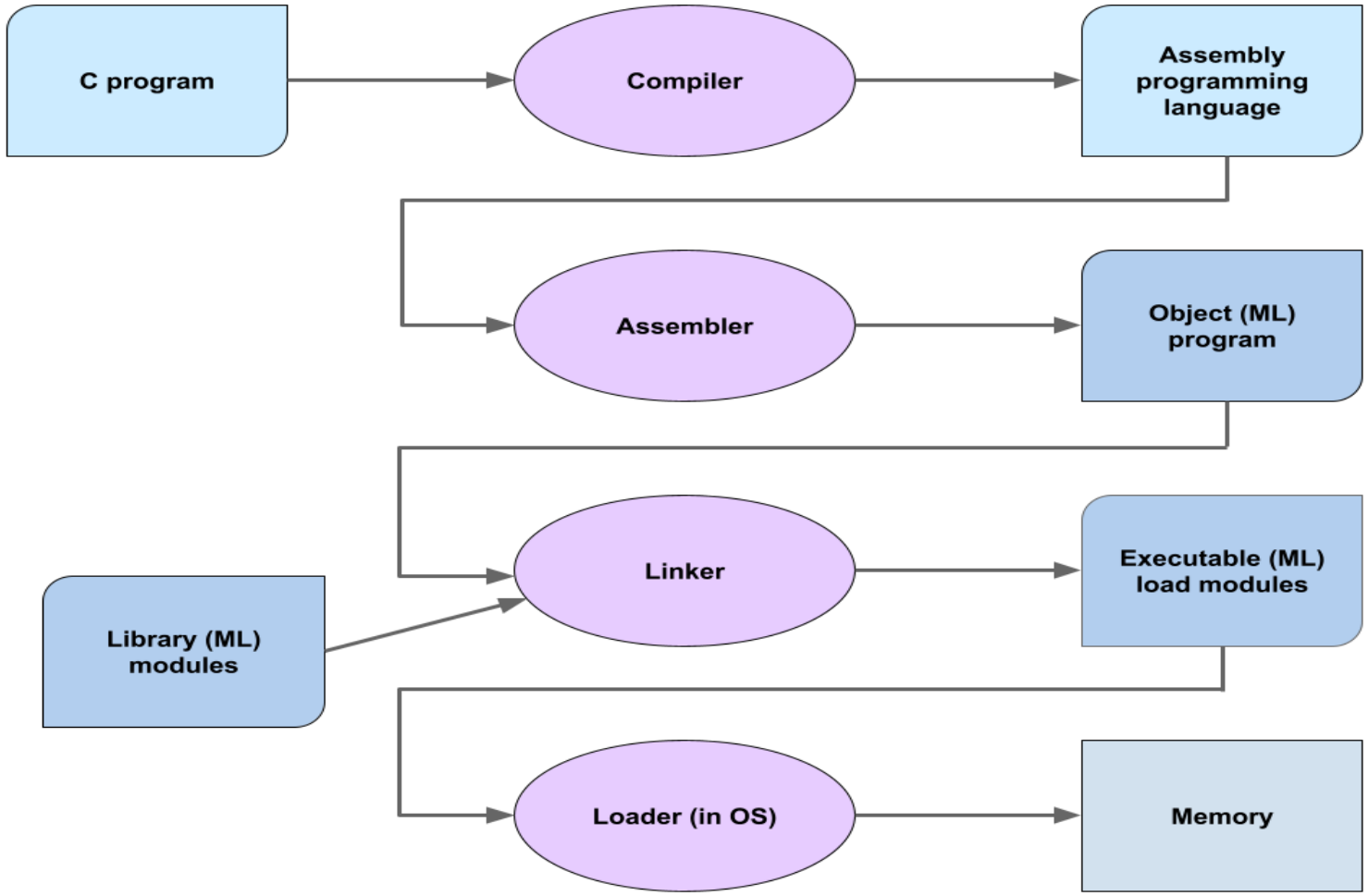
System Programming

- Various System Programs are:
  - Macro processors/ Pre-processors
  - Compilers
  - Interpreter
  - Assemblers
  - Linkers
  - Loaders

  - Operating System
  - Device drivers

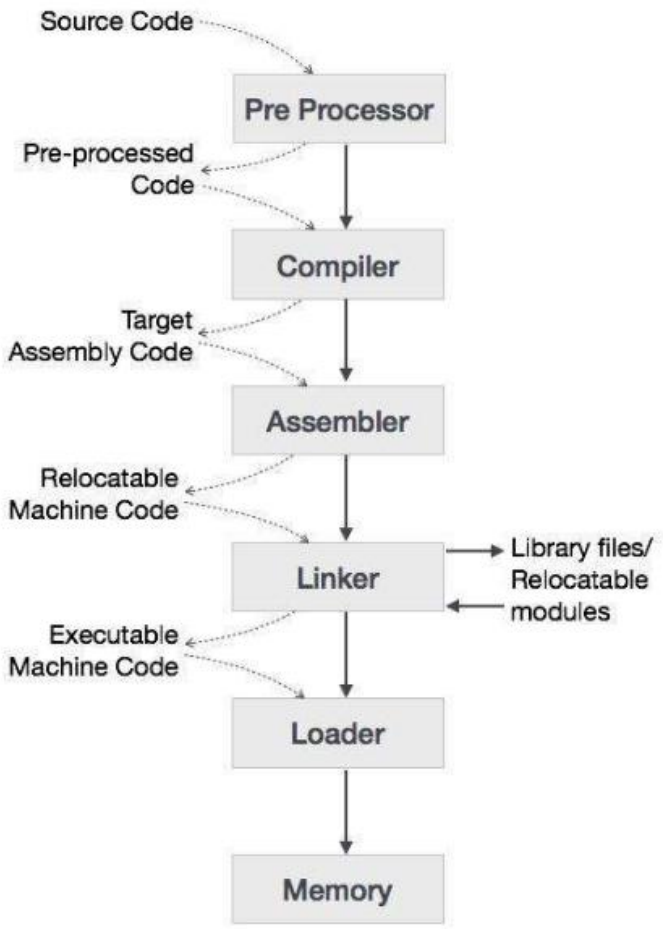
Somaiya TRUST

How computers translate HLL (C) to ML format



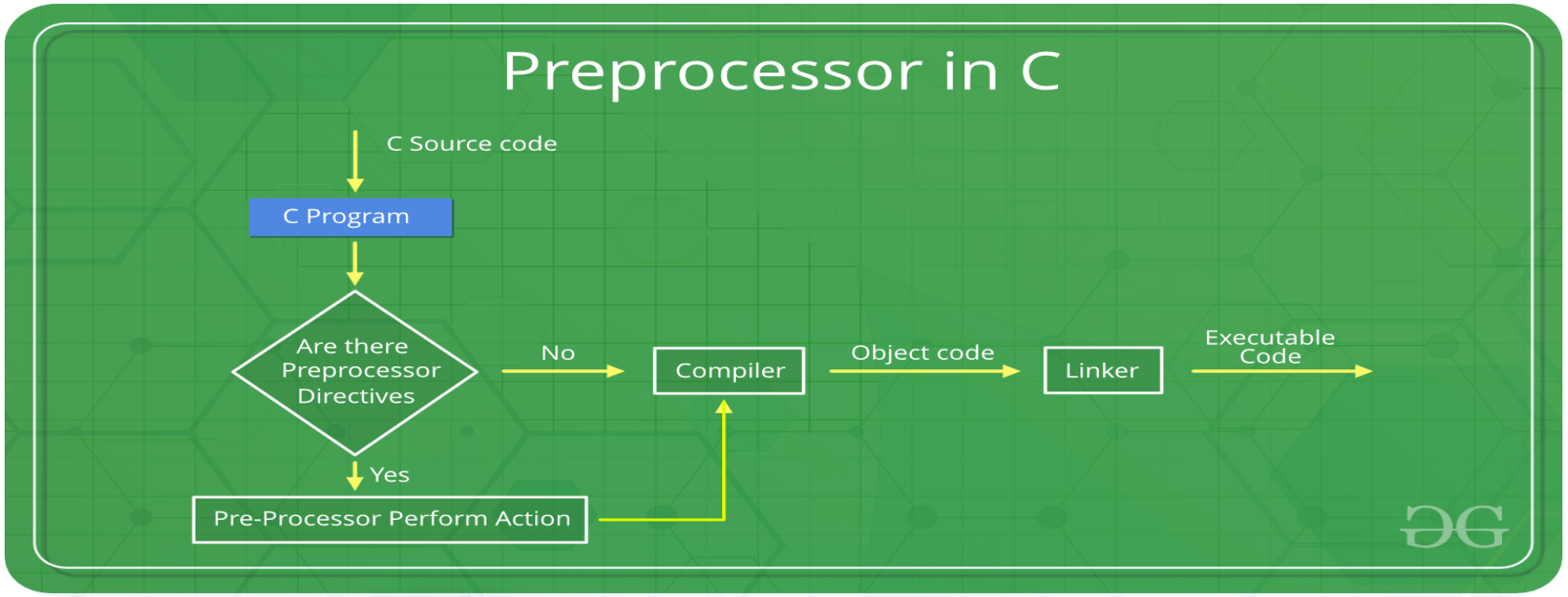
How a program, using C compiler, is executed on a host machine ?

- User writes a program in C language (high-level language)
- The C compiler, compiles the program and translates it to assembly program
- An assembler then translates the assembly program into machine code (object file)
- A linker is used to link all the parts of the program together for execution (executable machine code)
- A loader loads all of them into memory and then the program is executed



Macro Processors or Pre-Processors

- As the name suggests, Pre-processors are programs that process the source code before compilation
  - 4 main types of pre-processor directives:
  - Macros, File Inclusion, Conditional Compilation, Other directives



Macros

- Macros are a sequence of computing instructions available to the programmer as a single program statement
  - making the programming task less tedious and less error-prone
- It is a piece of code in a program, assigned some name
  - Whenever this name is encountered by the compiler, compiler replaces the name with actual piece of code
  - i.e It replaces the name with the definition of the macro
- The '#define' directive is used to define a macro
  - Macro definitions need not be terminated by semi-colon(;)

Macros

C program to illustrate macros

```
#include <stdio.h>
// Macro definition
#define LIMIT 5
int main()
{
        // Print the value of macro defined
        printf("The value of LIMIT is %d", LIMIT);
        return 0;
}
```

Output-

The value of LIMIT is 5

Macros

```
#include <stdio.h>

// macro definition
#define LIMIT 5

int main()
{
    for (int i = 0; i < LIMIT; i++) {
        printf("%d \n", i);
    }
    return 0;
}
```

Output??

File Inclusion

- This pre-processor directive tells the compiler to include a file in the source code/program

- There are 2 types of files which can be included by the user in the program:
  - **Header Files or Standard files**
  - **User defined files**

File Inclusion

Header File or Standard files:

- These files contains definition of pre-defined functions e.g. printf(), scanf() etc.; must be included for working with these functions.

- Different functions are declared in different header files
  - (E.g) C++ standard I/O functions are in 'iostream' header file whereas,
  - Functions which perform string operations are in 'string' header file

- **Syntax**: **#include< *file_name* >**
  - where *file_name* is the name of file to be included
  - The '<' and '>' brackets tells the compiler to look for the file in standard directory

File Inclusion

User defined files:

- When a program becomes very large, it is better to divide it into smaller files and include it whenever needed.
- These types of files are user defined files.

- These files can be included as: **#include"*filename*"**

File Inclusion

- **#include<filename.h>**

when we are including header file using this syntax then it will be loaded from default directory
i.e. C:\TC\INCLUDE

By using this syntax we are including pre-defined header files

#include “filename.h”

when we are including header file using this syntax then it will be loaded from the current working directory

By using this syntax we are including user-defined header files

Conditional Compilation Directives

- These are type of directives which help -
  - **To compile a specific portion of the program**
  - <u>or</u>
  - To skip compilation of some specific part of the program (based on some conditions)

- Can be done with the help of two pre-processing commands '**ifdef**' and '**endif**'

Conditional Compilation Directives

Syntax:

```
#ifdef macro_name
statement1;
statement2;
statement3;
.
.
.
statementN;
#endif
```

- If the macro with name as '*macroname*' is defined
  - **Then the block of statements will execute normally**
  - **Else the compiler will simply skip this block of statements**

Somaiya TRUST

Conditional Compilation Directives

```
#include <stdio.h>
#define YEARS_OLD 10
int main()
{
#ifdef YEARS_OLD
printf("TechOnTheNet is over %d years old.\n", YEARS_OLD);
#endif
printf("TechOnTheNet is a great resource.\n");
return 0;
}
```

Output

```
/tmp/PG7rWF95Ud.o
TechOnTheNet is over 10 years old.
TechOnTheNet is a great resource.
```

- In the C Programming Language, the #ifdef directive allows for conditional compilation.
- The preprocessor determines if the provided macro exists before including the subsequent code in the compilation process.

Other Directives

- 2 more directives which are not commonly used:
  - #undef Directive
  - #pragma Directive
- **#undef Directive**:
  - The #undef directive is used to undefine an existing macro
  - This directive works as:
    - #undef YEARS_OLD - Using this statement will undefine the existing macro YEARS_OLD
    - After this statement every “#ifdef YEARS_OLD” statement will evaluate to false
- **#pragma Directive**:
  - This directive is a special purpose directive used to turn on or off some features
  - This type of directives are compiler-specific (vary from compiler to compiler)

Undef Directive

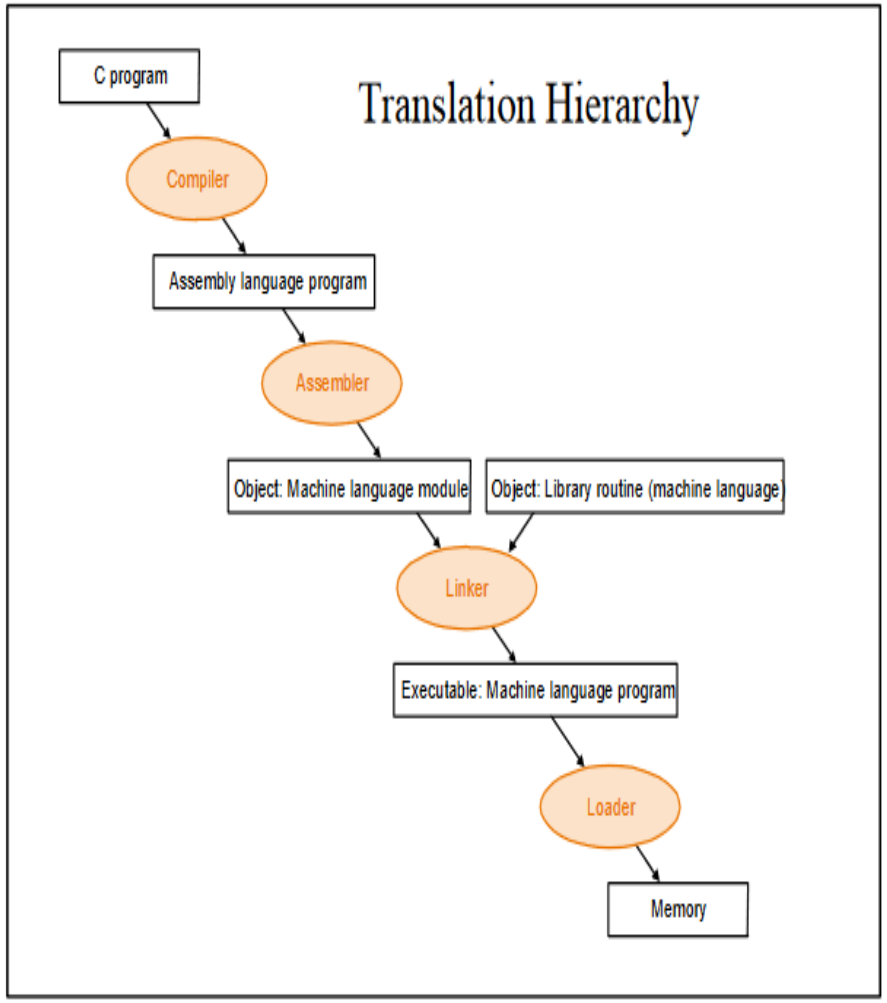
```
#include <stdio.h>
#define YEARS_OLD 10
int main()
{
------
#undef YEARS_OLD
#ifdef YEARS_OLD
printf("TechOnTheNet is over %d years old.\n", YEARS_OLD);
#endif
printf("TechOnTheNet is a great resource.\n");
return 0;
}
```

Output

Compiler

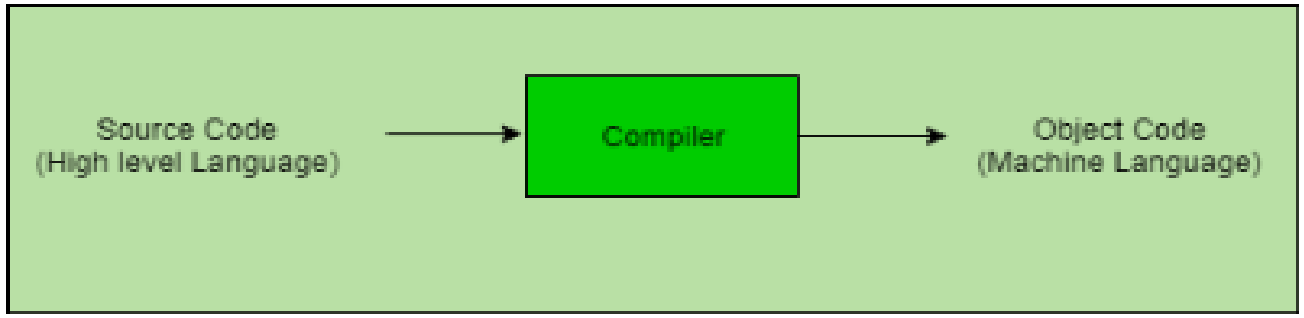
- A compiler is a software that:
  - translates the code written in one language to some other language, without changing the meaning of the program

- **"C" compiler converts high-level language program into assembly language**



Compiler

- **Compiler: Language processor** that reads the complete source program/code written in high level language **as a whole in one go and translates it** into an equivalent program in machine language

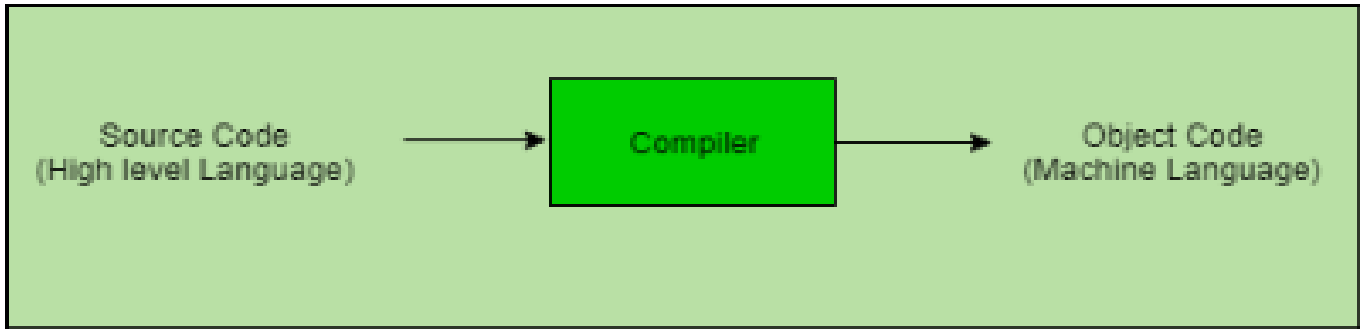


- The compiler also
  - makes the **target code efficient,** and
  - optimized in terms of time and space

Compiler

- **Source code is translated to object code** successfully if it is free of errors

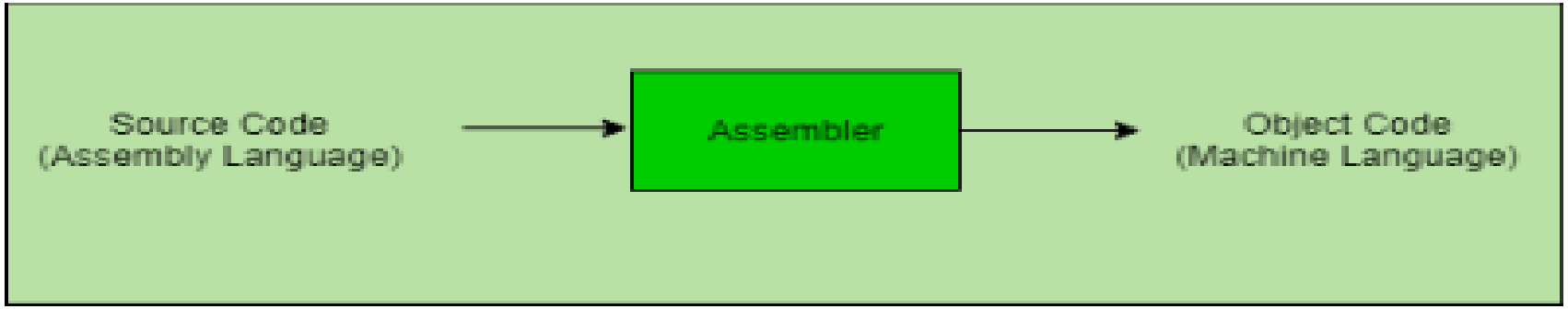
- Compiler **specifies the errors at the end of compilation with line numbers** (when there are any errors in the source code)
  - errors must be removed before the compiler can successfully recompile the source code again



Assembler

- A program that converts assembly language programs into object files

- Object files contain a combination of
  - machine instructions
  - data, and
  - information needed to place instructions properly in memory



Interpreter

- Interpreter, like a compiler, **translates high-level language into low-level machine language**

- **Difference** lies in the way they read the source code
  - A compiler reads the **whole source code at once; creates tokens, checks semantics, generates intermediate code, executes the whole program and may involve many passes**

  - An interpreter reads **a statement from the input, executes it, then takes the next statement in sequence**

Somaiya TRUST

Interpreter

- If an error occurs
  - **an interpreter stops execution and reports it;**
  - **It moves on to the next line for execution only after removal of the error**
  - whereas a compiler **reads the whole program even if it encounters several errors.**

- **Example:** Perl, Matlab, Python…

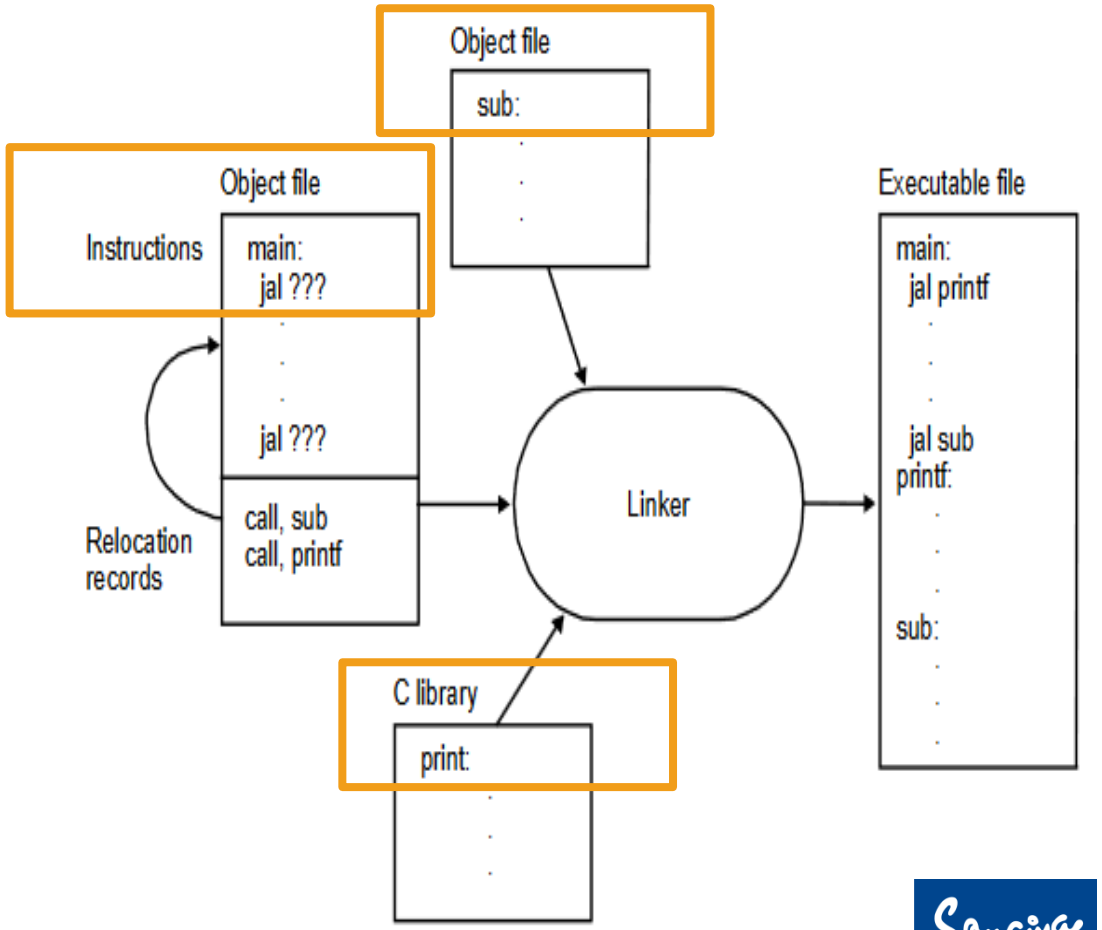
Somaiya TRUST

Compiler V/s Interpreter

COMPILER	INTERPRETER
A compiler is a program which coverts the entire source code of a programming language into executable machine code for a CPU.	interpreter takes a source program and runs it line by line, translating each line as it comes to it.
Compiler takes large amount of time to analyze the entire source code but the overall execution time of the program is comparatively faster.	Interpreter takes less amount of time to analyze the source code but the overall execution time of the program is slower.
Compiler generates the error message only after scanning the whole program, so debugging is comparatively hard as the error can be present any where in the program.	Its Debugging is easier as it continues translating the program until the error is met
Generates intermediate object code.	No intermediate object code is generated.
Examples: C, C++, Java	Examples: Python, Perl

Linker

- A linker is a tool used to **link all the parts of the program together for execution** (executable machine code)
- **Merges the object files** produced by separate compilation or assembly and **creates a single executable file**
- Linker also called as <u>link editor</u>
-



Linker

- Major tasks–
  - **<u>search and locate referenced module/routines</u>** in a program
  - **Searches the program to find <u>library routines</u> used by program,**
  - **e.g. printf(), math routines,...**

Loader in C/C++

- **Loader** is the program that **loads the executable from the disk into the primary memory(RAM) for execution**.

- **Allocates space** to the executable module **in main memory** and then transfers control to the beginning instruction of the program

Loader in C/C++

- Loading a program involves:
  - **reading the contents of the executable file containing the program instructions into memory,**
  - carrying out other required **preparatory tasks to prepare the executable for running**

- Once loading is complete, the operating system starts the program by passing control to the loaded program code.

Somaiya TRUST

Questions to Test Your Understanding…

GATE CS 1998 | Question 25

In a resident-OS computer, which of the following system software must reside in the main memory under all situations?

(A) Assembler

(B) Linker

(C) Loader

(D) Compiler

GATE CS 1998 | Question 25

In a resident- OS computer, which of the following system software must reside in the main memory under all situations?

(A) Assembler
(B) Linker
(C) Loader
(D) Compiler

Answer: (C)

Explanation: Loader is the part of an operating system that is responsible for loading programs and libraries. It places the programs into memory and also prepares them for execution.

Loading a program involves tasks such as reading the contents of the executable file containing the program instructions into memory, and then carrying out other required preparatory tasks to prepare the executable for running so any software must reside in the main memory under all situations.

The operating system starts the program by passing control to the loaded program code once the loading process is completed.

Option (C) is correct.

Q) A linker program

a. places the program in the memory for the purpose of execution.

b. relocates the program to execute from the specific memory area allocated to it.

c. links the program with other programs needed for its execution.

d. interfaces the program with the entities generating its input data.

Thank-You

Lets start with Operating System basics!